Technical Documentation

QEMU System Launcher

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

BMW Car IT, Ulm

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

1. Introduction

1.1. Purpose

This document provides technical specifications for the launch_qemu.sh script, a dedicated launcher used to boot a Linux kernel within the QEMU emulator. Its primary function is to facilitate the integrity testing workflow by providing an interactive menu to select and boot either the original, working root filesystem or one of the corrupted test images created by the integrity_tests.sh suite.

The script ensures the kernel is launched with the correct parameters to enable dm-verity verification during the initial boot phase.

1.2. Testing Workflow

The launcher is the final step in the dm-verity validation process:

- Workflow Step 1: Create test images using `./integrity_tests.sh <mode>`.
- Workflow Step 2: Execute `./launch_qemu.sh`.
- Workflow Step 3: Select a test image from the generated menu.
- Workflow Step 4: Observe the kernel's serial output (ttyS0) to verify the expected security failure or successful boot.

1.3. Location and Dependencies

The script automatically locates the necessary components:

- Kernel Image: `$SCRIPT\_DIR/../bootloaders/kernel\_image.bin`
- Original Image: `$SCRIPT\_DIR/Binaries/rootfs.img`
- Test Images: All files matching `rootfs.*.test.img` found in the `$SCRIPT\_DIR/Binaries directory`.

2. Script Specification: launch_qemu.sh

2.1. System and Configuration

The script uses standard Bash syntax with strict error handling (`set -eo pipefail`). It requires the qemu-system-x86_64 binary to be installed in the environment's PATH.

The launch configuration is standardized:

- Architecture: x86_64
- Memory: 1024MB
- Console: Serial (`-nographic -append console=ttyS0`)
- Disk Interface: `virtio-blk-pci` (disk ID hd0)

2.2. Kernel Command Line Parameters

The script constructs the critical kernel append command (`$APPEND\_CMD`) necessary for the custom dm-verity autoboot module:

```
52  # Kernel parameters for dm-verity testing
53  APPEND_CMD="console=ttyS0 loglevel=7 root=/dev/dm-0 rootfstype=ext4 rootwait \
54  dm_verity_autoboot.autoboot_device=/dev/vda \
55  dm_verity_autoboot.mode=verify_and_map"
```

Key parameters in the command line:

- `root=/dev/dm-0`: Instructs the kernel to mount the root filesystem from the dm-verity device mapper target, not the raw disk.
- `dm\_verity\_autoboot.autoboot\_device=/dev/vda`: Specifies the raw disk image (/dev/vda in QEMU) containing the dm-verity metadata (the VLOC footer) to be verified.
- `dm\_verity\_autoboot.mode=verify\_and\_map`: Tells the custom module to perform the verification and, if successful, map the block device as /dev/dm-0.

2.3. Image Selection and Logic

The script dynamically builds a numbered menu based on available images:

- Original Image: Always option 1 (if it exists).
- Test Images: Numbered sequentially following the original image, with a descriptive label extracted from the filename (e.g., `rootfs.sig1.test.img` is labeled as sig1 test image).

The selection logic processes the user's input ($choice) and sets the final `$ROOTFS` path for the QEMU command. Default behavior is to select the original image, or the first test image if the original is missing.

2.4. QEMU Execution Command

The script uses exec qemu-system-x86_64 to launch the virtual machine, replacing the current shell process with the QEMU application itself.

The final QEMU command uses the following structure:

```
206  exec qemu-system-x86_64 \
207    -kernel "$KERNEL" \
208    -drive if=none,file="$ROOTFS",format=raw,id=hd0 \
209    -device virtio-blk-pci,drive=hd0 \
210    -append "$APPEND_CMD" \
211    -m 1024M -nographic
```

End of Document